

Quicksort

Not in-place

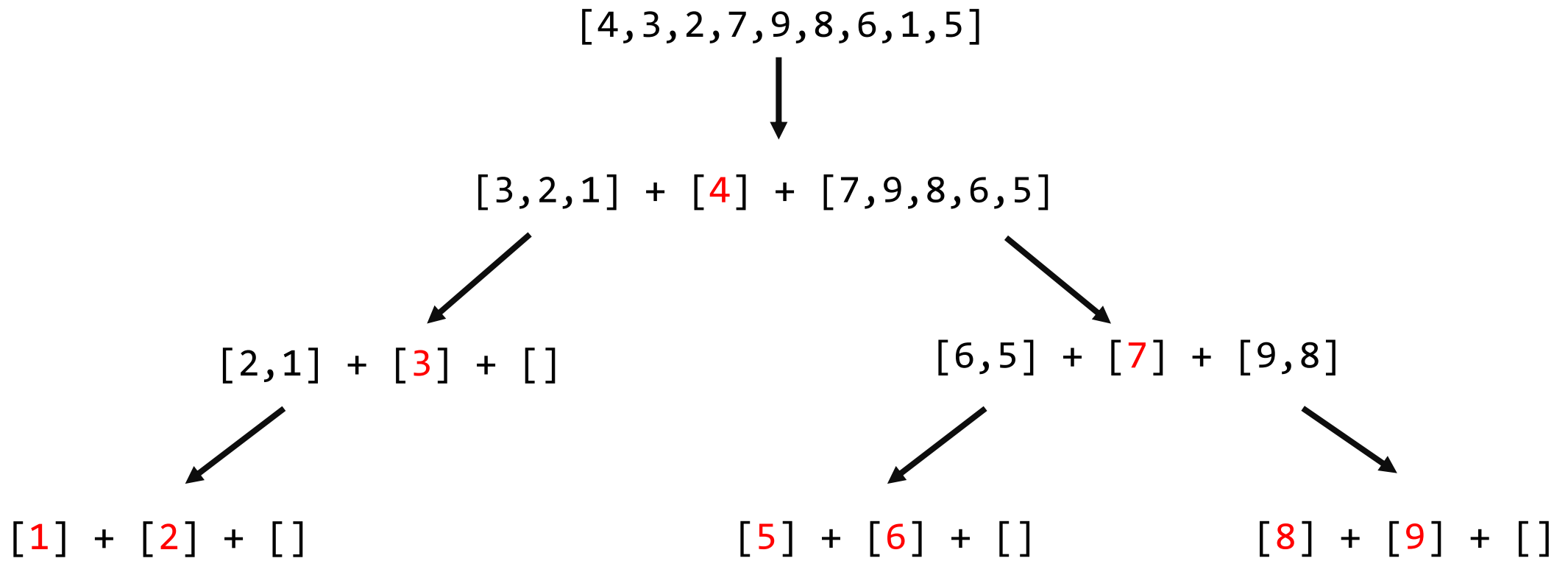
Recursive Call Trace

Main Idea

- Divide and Conquer
- Choose a pivot item from the list and divide the list into two new lists
 - items less than and equal to the pivot are placed on the left side
 - items greater than the pivot are placed on the right.
- We repeat the above process for the resulting left and right lists until we end up with a list of size 1.
- At this stage, all elements are already sorted, and we combine all the results to return the final sorted list.

```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```

Recursive Call Trace



Quicksort we evaluate the following tree, it does this using depth-first, this means it will evaluate to the left until it hits a base-case.

The following shows how the recursive calls are evaluated

```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```

[4,3,2,7,9,8,6,1,5]

```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```

[4,3,2,7,9,8,6,1,5]


```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```

[4,3,2,7,9,8,6,1,5]

```
def quicksort(items):
    if len(items) <= 1: # base case - single item in list
        return items

    pivot = items[0] # choose pivot as first item
    low = []
    high = []

    # sort the items into low or high list
    for i in range(1, len(items)):
        if items[i] < pivot:
            low.append(items[i])
        else:
            high.append(items[i])

    # recursively call quicksort on lower and higher
    # return combined result
    return quicksort(low) + [pivot] + quicksort(high)

list1 = [4,3,2,7,9,8,6,1,5]
quicksort(list1)
```

[4,3,2,7,9,8,6,1,5]

```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```

[4,3,2,7,9,8,6,1,5]



[3,2,1]

```
def quicksort(items):
    if len(items) <= 1: # base case - single item in list
        return items

    pivot = items[0] # choose pivot as first item
    low = []
    high = []

    # sort the items into low or high list
    for i in range(1, len(items)):
        if items[i] < pivot:
            low.append(items[i])
        else:
            high.append(items[i])

    # recursively call quicksort on lower and higher
    # return combined result
    return quicksort(low) + [pivot] + quicksort(high)

list1 = [4,3,2,7,9,8,6,1,5]
quicksort(list1)
```

[4,3,2,7,9,8,6,1,5]



[3,2,1]

```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```

[4,3,2,7,9,8,6,1,5]



[3,2,1]



[2,1]

```
def quicksort(items):
    if len(items) <= 1: # base case - single item in list
        return items

    pivot = items[0] # choose pivot as first item
    low = []
    high = []

    # sort the items into low or high list
    for i in range(1, len(items)):
        if items[i] < pivot:
            low.append(items[i])
        else:
            high.append(items[i])

    # recursively call quicksort on lower and higher
    # return combined result
    return quicksort(low) + [pivot] + quicksort(high)

list1 = [4,3,2,7,9,8,6,1,5]
quicksort(list1)
```

[4,3,2,7,9,8,6,1,5]



[3,2,1]



[2,1]

```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```

[4,3,2,7,9,8,6,1,5]



[3,2,1]



[2,1]



[1]

```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```

[4,3,2,7,9,8,6,1,5]



[3,2,1]



[2,1]



[1]


```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```

[4,3,2,7,9,8,6,1,5]



[3,2,1]



[2,1]



[1] + [2]

```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```

[4,3,2,7,9,8,6,1,5]



[3,2,1]



[2,1]



[1] + [2]

```

def quicksort(items):
    if len(items) <= 1: # base case - single item in list
        return items

    pivot = items[0] # choose pivot as first item
    low = []
    high = []

    # sort the items into low or high list
    for i in range(1, len(items)):
        if items[i] < pivot:
            low.append(items[i])
        else:
            high.append(items[i])

    # recursively call quicksort on lower and higher
    # return combined result
    return quicksort(low) + [pivot] + quicksort(high)

list1 = [4,3,2,7,9,8,6,1,5]
quicksort(list1)

```

[4,3,2,7,9,8,6,1,5]



[3,2,1]



[2,1]



[1] + [2] + []

```

def quicksort(items):
    if len(items) <= 1: # base case - single item in list
        return items

    pivot = items[0] # choose pivot as first item
    low = []
    high = []

    # sort the items into low or high list
    for i in range(1, len(items)):
        if items[i] < pivot:
            low.append(items[i])
        else:
            high.append(items[i])

    # recursively call quicksort on lower and higher
    # return combined result
    return quicksort(low) + [pivot] + quicksort(high)

list1 = [4,3,2,7,9,8,6,1,5]
quicksort(list1)

```

[4,3,2,7,9,8,6,1,5]



[3,2,1]



[2,1]



[1] + [2] + []

```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```

[1,2]

[4,3,2,7,9,8,6,1,5]



[3,2,1]



```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```

[4,3,2,7,9,8,6,1,5]



[3,2,1]



[1,2] + [3]

```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```

[4,3,2,7,9,8,6,1,5]



[3,2,1]



[1,2] + [3]

```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```

[4,3,2,7,9,8,6,1,5]



[3,2,1]



[1,2] + [3] + []


```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```

[4,3,2,7,9,8,6,1,5]



[3,2,1]



[1,2] + [3] + []

```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```

[4,3,2,7,9,8,6,1,5]



[1,2,3]

```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```

[4,3,2,7,9,8,6,1,5]
↓
[1,2,3] + [4]

```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```

[4,3,2,7,9,8,6,1,5]
↓
[1,2,3] + [4]

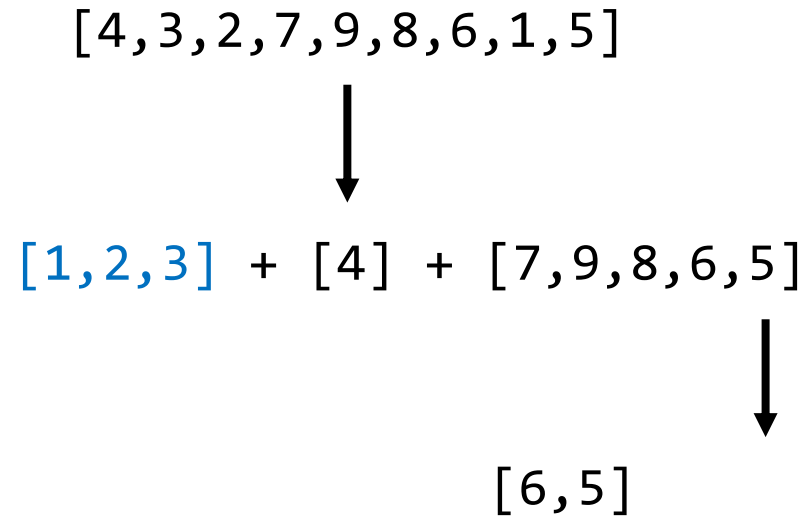
```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```

[4,3,2,7,9,8,6,1,5]
↓
[1,2,3] + [4] + [7,9,8,6,5]

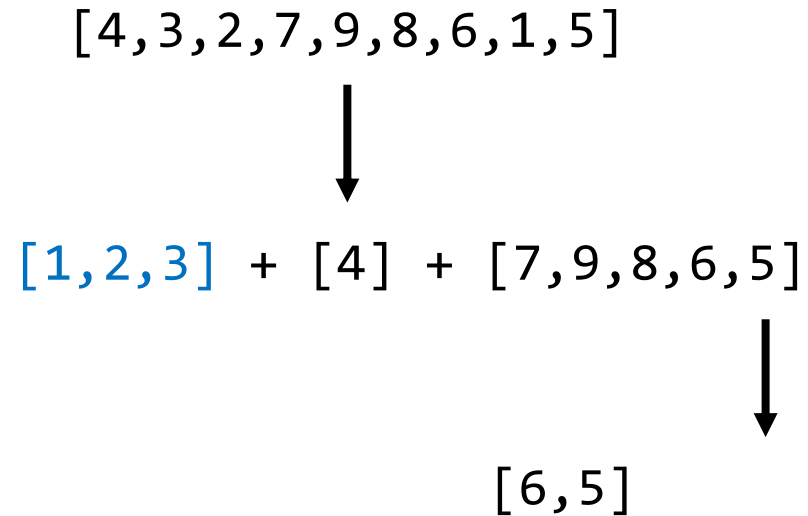
```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```

[4,3,2,7,9,8,6,1,5]
↓
[1,2,3] + [4] + [7,9,8,6,5]

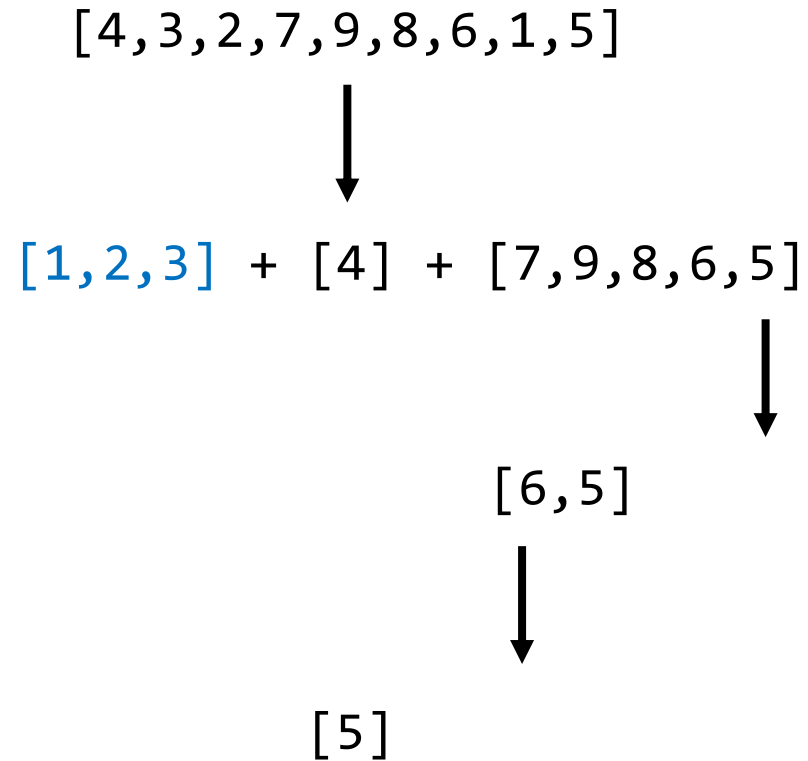
```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```



```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```




```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```



```

def quicksort(items):
    if len(items) <= 1: # base case - single item in list
        return items

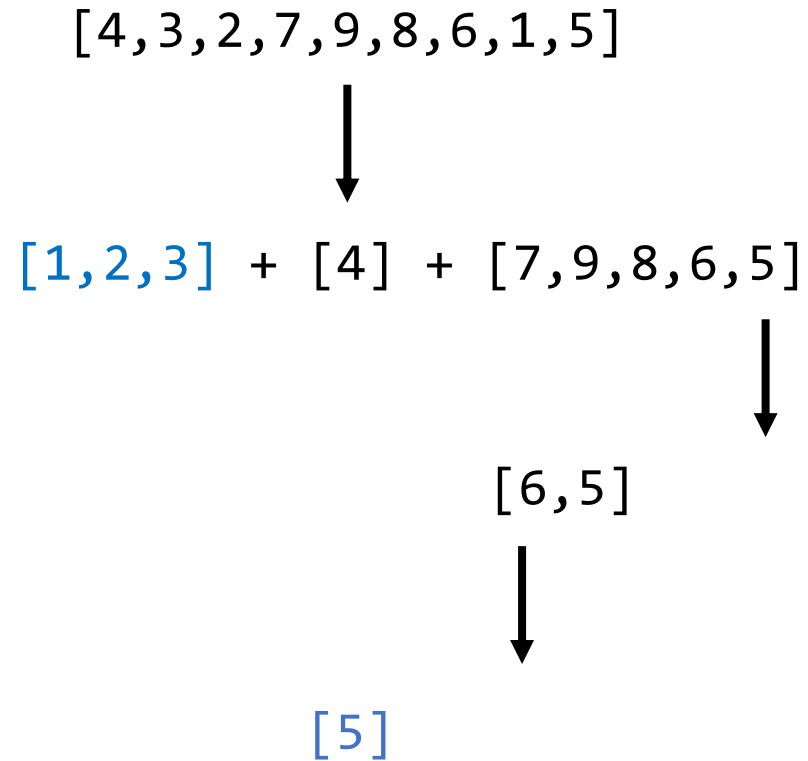
    pivot = items[0] # choose pivot as first item
    low = []
    high = []

    # sort the items into low or high list
    for i in range(1, len(items)):
        if items[i] < pivot:
            low.append(items[i])
        else:
            high.append(items[i])

    # recursively call quicksort on lower and higher
    # return combined result
    return quicksort(low) + [pivot] + quicksort(high)

list1 = [4,3,2,7,9,8,6,1,5]
quicksort(list1)

```



```

def quicksort(items):
    if len(items) <= 1: # base case - single item in list
        return items

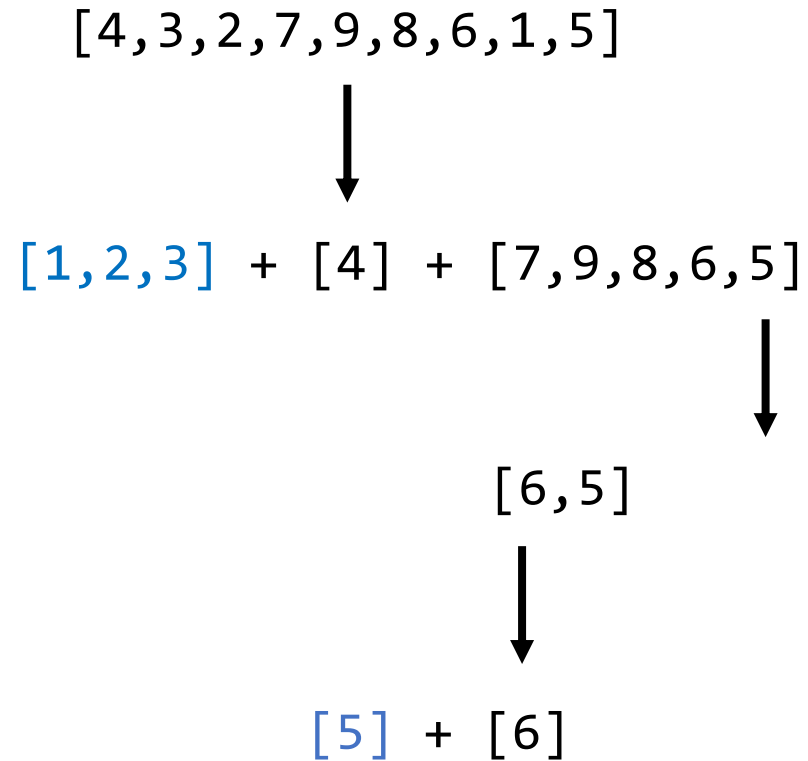
    pivot = items[0] # choose pivot as first item
    low = []
    high = []

    # sort the items into low or high list
    for i in range(1, len(items)):
        if items[i] < pivot:
            low.append(items[i])
        else:
            high.append(items[i])

    # recursively call quicksort on lower and higher
    # return combined result
    return quicksort(low) + [pivot] + quicksort(high)

list1 = [4,3,2,7,9,8,6,1,5]
quicksort(list1)

```



```

def quicksort(items):
    if len(items) <= 1: # base case - single item in list
        return items

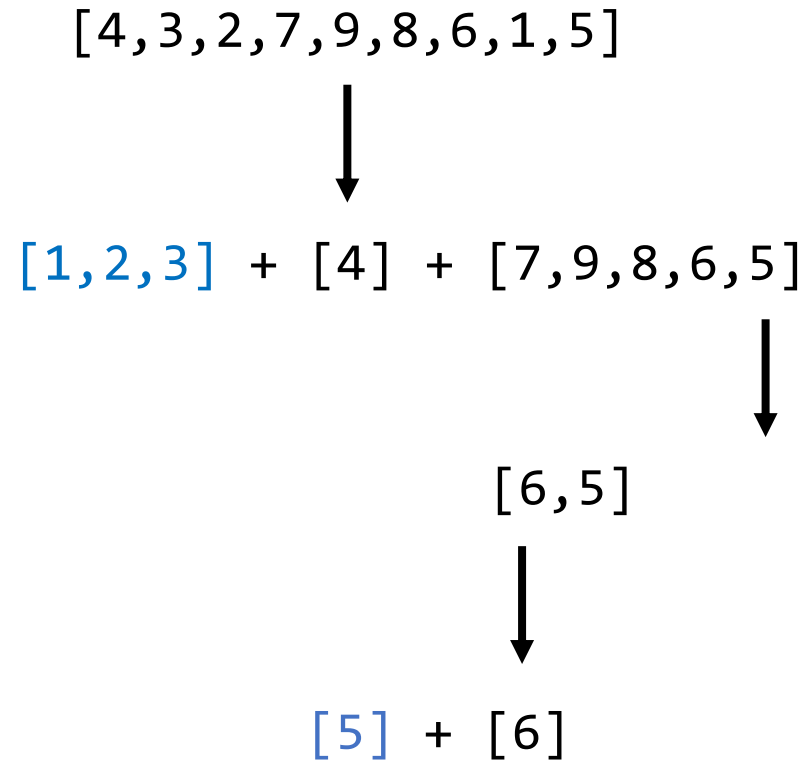
    pivot = items[0] # choose pivot as first item
    low = []
    high = []

    # sort the items into low or high list
    for i in range(1, len(items)):
        if items[i] < pivot:
            low.append(items[i])
        else:
            high.append(items[i])

    # recursively call quicksort on lower and higher
    # return combined result
    return quicksort(low) + [pivot] + quicksort(high)

list1 = [4,3,2,7,9,8,6,1,5]
quicksort(list1)

```



```

def quicksort(items):
    if len(items) <= 1: # base case - single item in list
        return items

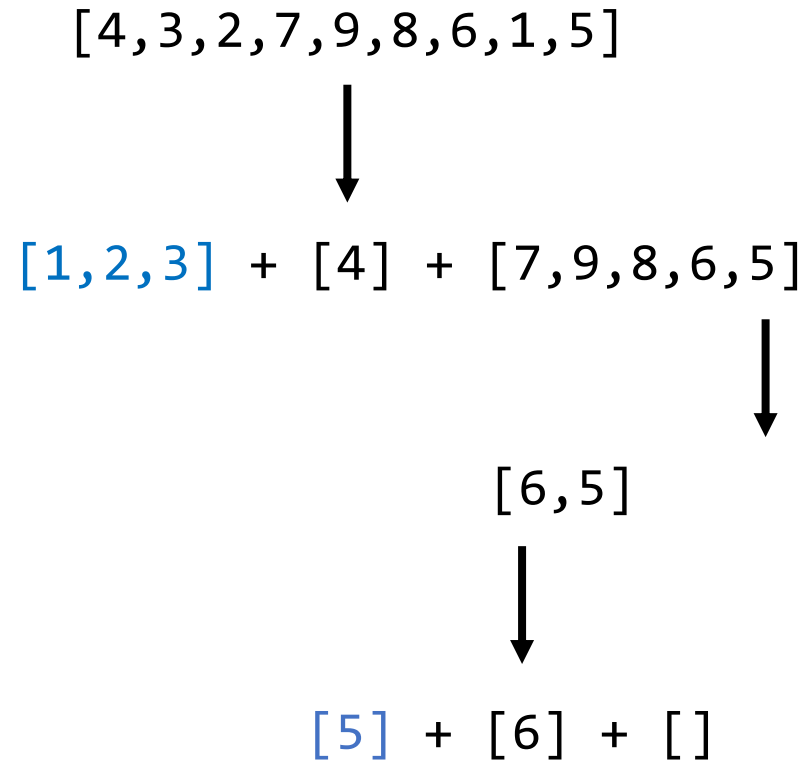
    pivot = items[0] # choose pivot as first item
    low = []
    high = []

    # sort the items into low or high list
    for i in range(1, len(items)):
        if items[i] < pivot:
            low.append(items[i])
        else:
            high.append(items[i])

    # recursively call quicksort on lower and higher
    # return combined result
    return quicksort(low) + [pivot] + quicksort(high)

list1 = [4,3,2,7,9,8,6,1,5]
quicksort(list1)

```



```

def quicksort(items):
    if len(items) <= 1: # base case - single item in list
        return items

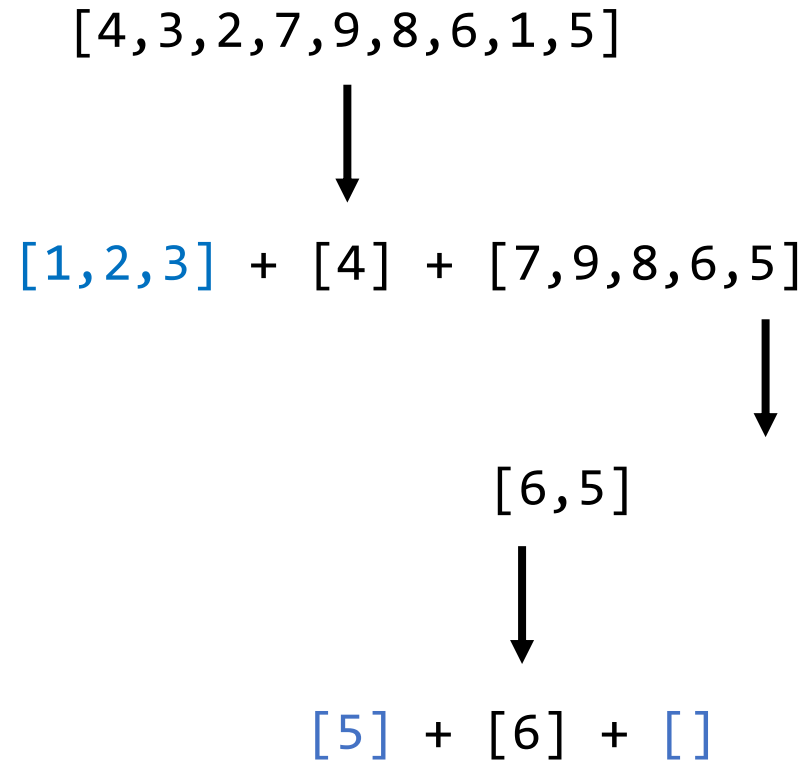
    pivot = items[0] # choose pivot as first item
    low = []
    high = []

    # sort the items into low or high list
    for i in range(1, len(items)):
        if items[i] < pivot:
            low.append(items[i])
        else:
            high.append(items[i])

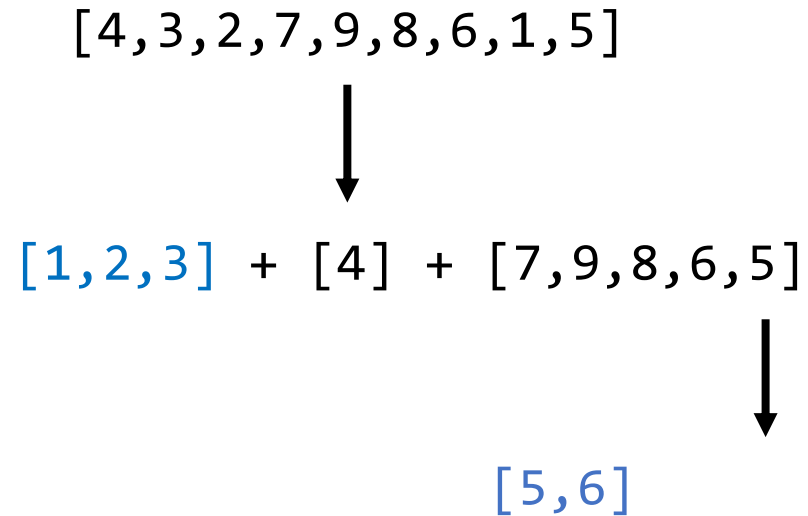
    # recursively call quicksort on lower and higher
    # return combined result
    return quicksort(low) + [pivot] + quicksort(high)

list1 = [4,3,2,7,9,8,6,1,5]
quicksort(list1)

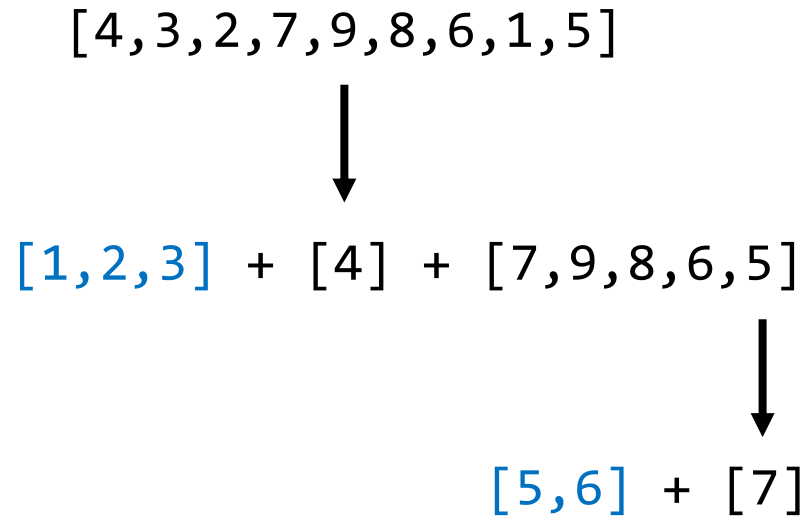
```



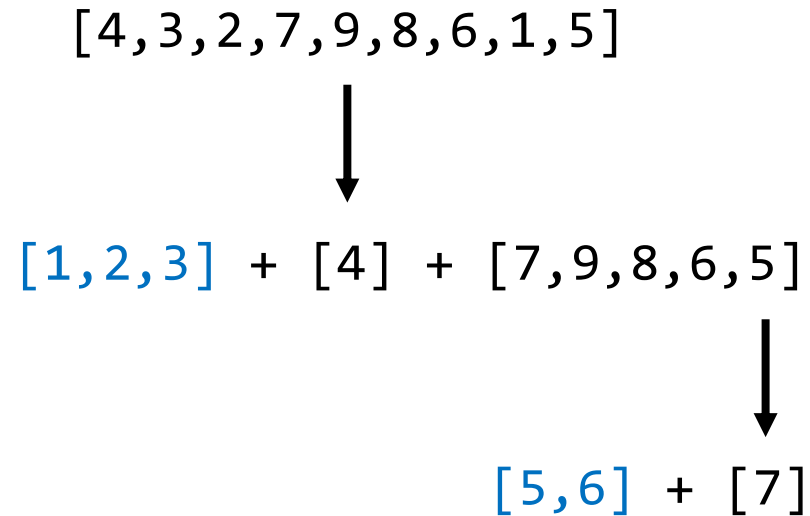
```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```



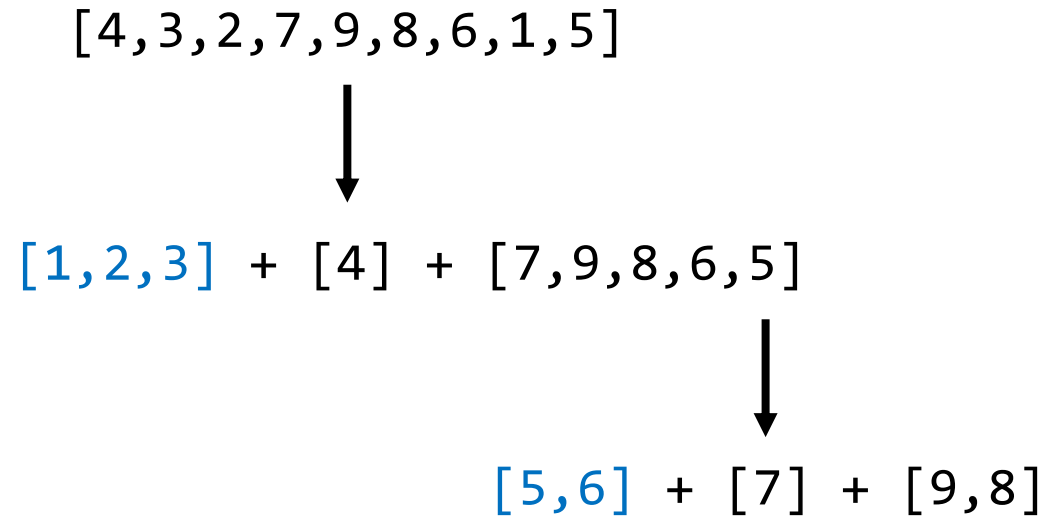
```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```




```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```



```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```



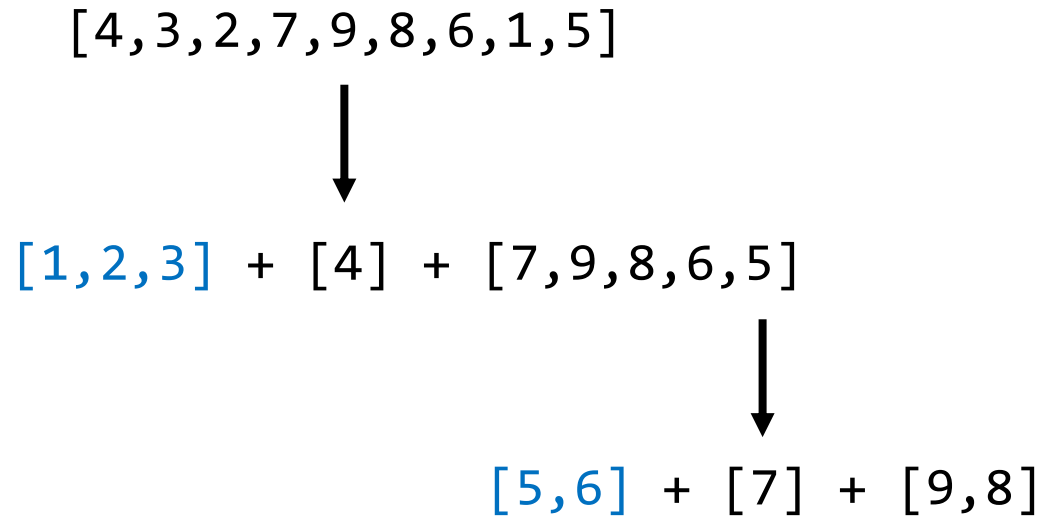
```
def quicksort(items):
    if len(items) <= 1: # base case - single item in list
        return items

    pivot = items[0] # choose pivot as first item
    low = []
    high = []

    # sort the items into low or high list
    for i in range(1, len(items)):
        if items[i] < pivot:
            low.append(items[i])
        else:
            high.append(items[i])

    # recursively call quicksort on lower and higher
    # return combined result
    return quicksort(low) + [pivot] + quicksort(high)

list1 = [4,3,2,7,9,8,6,1,5]
quicksort(list1)
```



```

def quicksort(items):
    if len(items) <= 1: # base case - single item in list
        return items

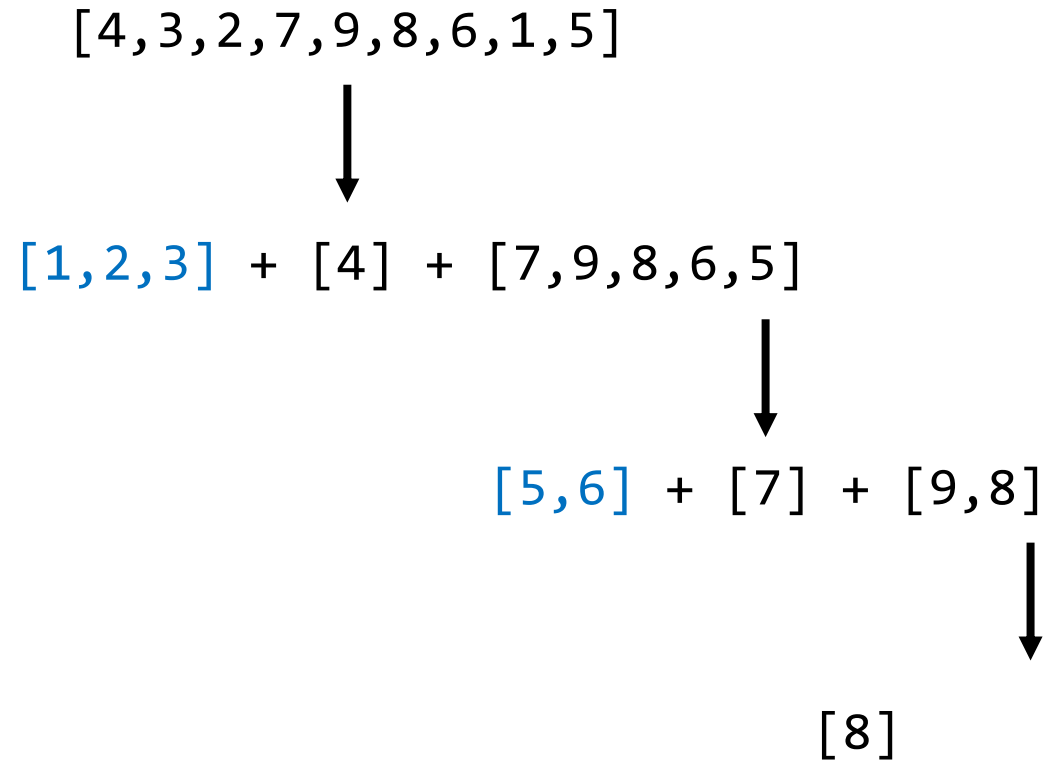
    pivot = items[0] # choose pivot as first item
    low = []
    high = []

    # sort the items into low or high list
    for i in range(1, len(items)):
        if items[i] < pivot:
            low.append(items[i])
        else:
            high.append(items[i])

    # recursively call quicksort on lower and higher
    # return combined result
    return quicksort(low) + [pivot] + quicksort(high)

list1 = [4,3,2,7,9,8,6,1,5]
quicksort(list1)

```



```

def quicksort(items):
    if len(items) <= 1: # base case - single item in list
        return items

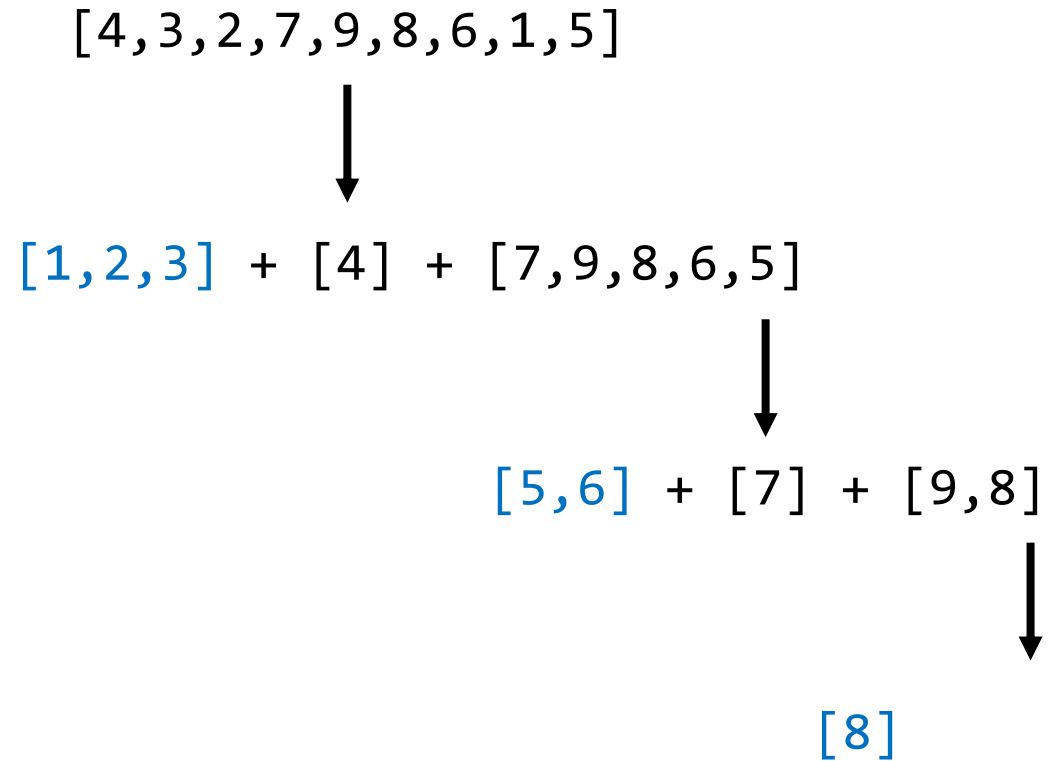
    pivot = items[0] # choose pivot as first item
    low = []
    high = []

    # sort the items into low or high list
    for i in range(1, len(items)):
        if items[i] < pivot:
            low.append(items[i])
        else:
            high.append(items[i])

    # recursively call quicksort on lower and higher
    # return combined result
    return quicksort(low) + [pivot] + quicksort(high)

list1 = [4,3,2,7,9,8,6,1,5]
quicksort(list1)

```



```

def quicksort(items):
    if len(items) <= 1: # base case - single item in list
        return items

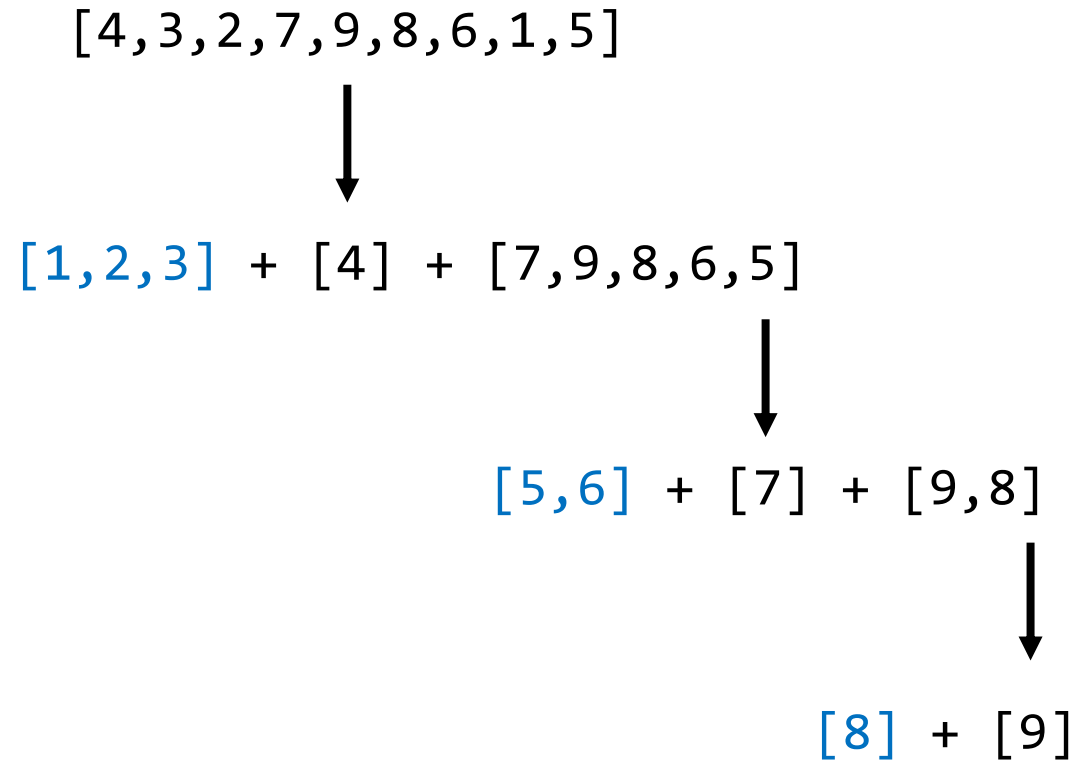
    pivot = items[0] # choose pivot as first item
    low = []
    high = []

    # sort the items into low or high list
    for i in range(1, len(items)):
        if items[i] < pivot:
            low.append(items[i])
        else:
            high.append(items[i])

    # recursively call quicksort on lower and higher
    # return combined result
    return quicksort(low) + [pivot] + quicksort(high)

list1 = [4,3,2,7,9,8,6,1,5]
quicksort(list1)

```



```

def quicksort(items):
    if len(items) <= 1: # base case - single item in list
        return items

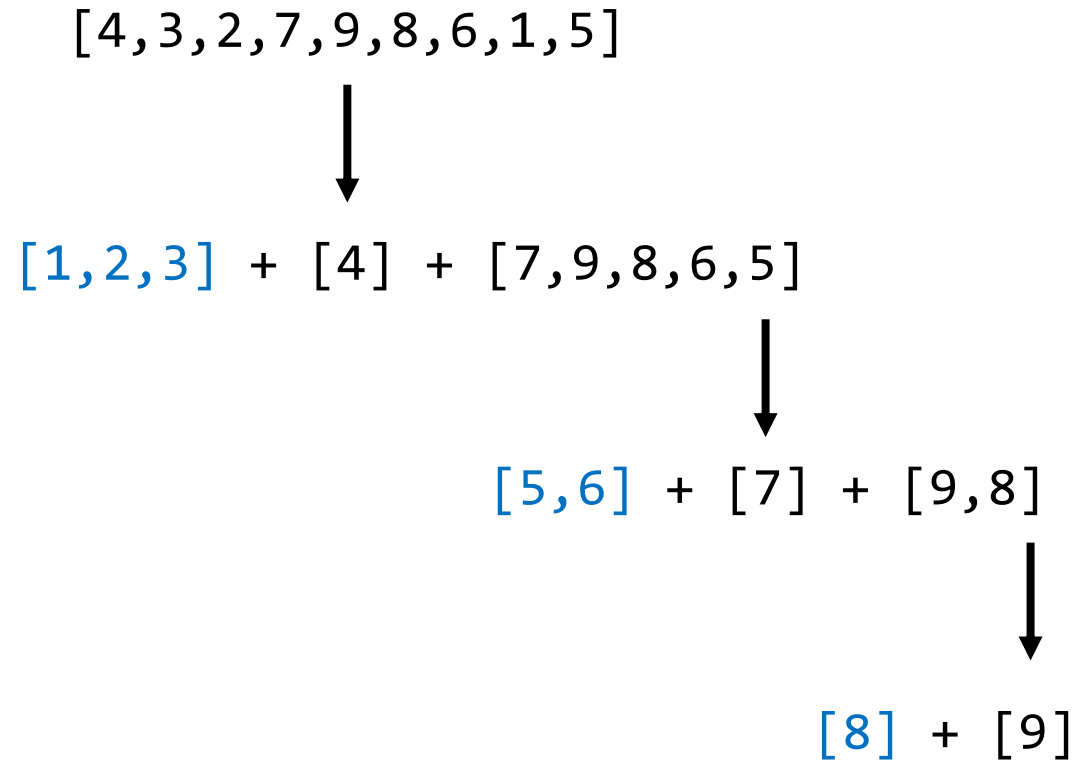
    pivot = items[0] # choose pivot as first item
    low = []
    high = []

    # sort the items into low or high list
    for i in range(1, len(items)):
        if items[i] < pivot:
            low.append(items[i])
        else:
            high.append(items[i])

    # recursively call quicksort on lower and higher
    # return combined result
    return quicksort(low) + [pivot] + quicksort(high)

list1 = [4,3,2,7,9,8,6,1,5]
quicksort(list1)

```



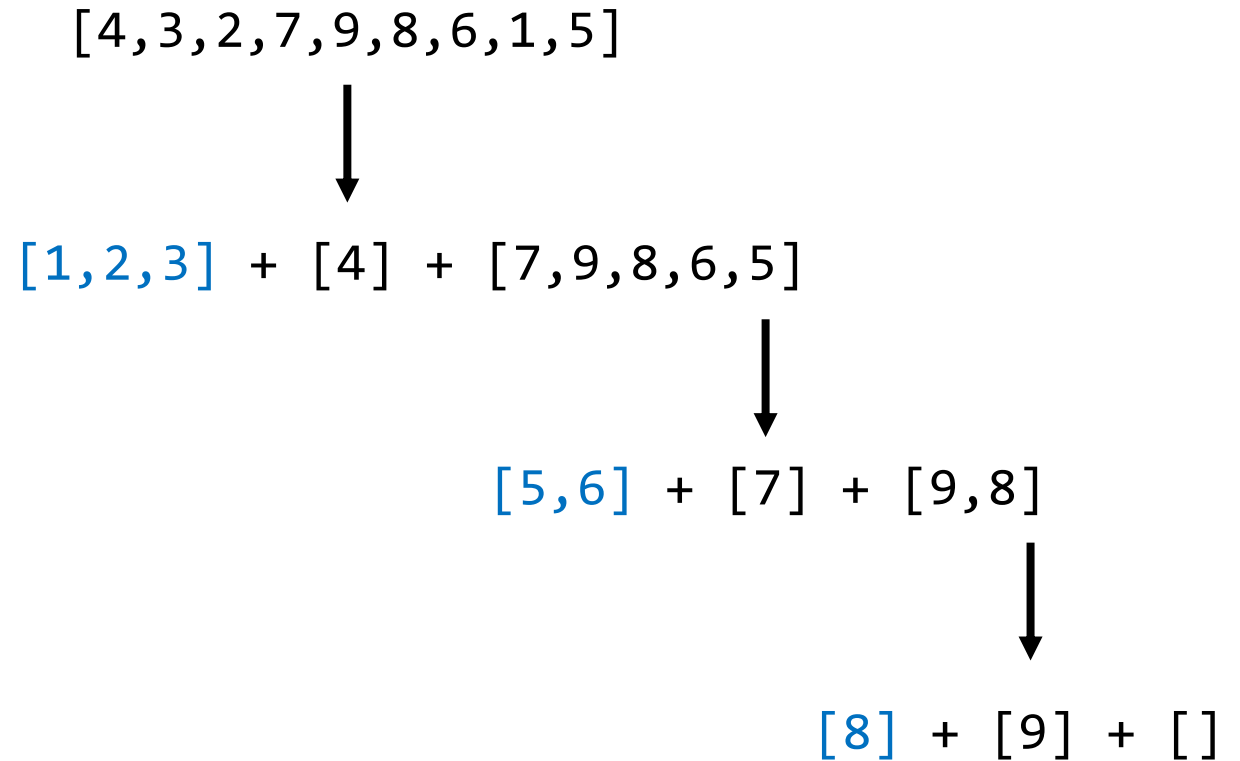
```
def quicksort(items):
    if len(items) <= 1: # base case - single item in list
        return items

    pivot = items[0] # choose pivot as first item
    low = []
    high = []

    # sort the items into low or high list
    for i in range(1, len(items)):
        if items[i] < pivot:
            low.append(items[i])
        else:
            high.append(items[i])

    # recursively call quicksort on lower and higher
    # return combined result
    return quicksort(low) + [pivot] + quicksort(high)

list1 = [4,3,2,7,9,8,6,1,5]
quicksort(list1)
```




```

def quicksort(items):
    if len(items) <= 1: # base case - single item in list
        return items

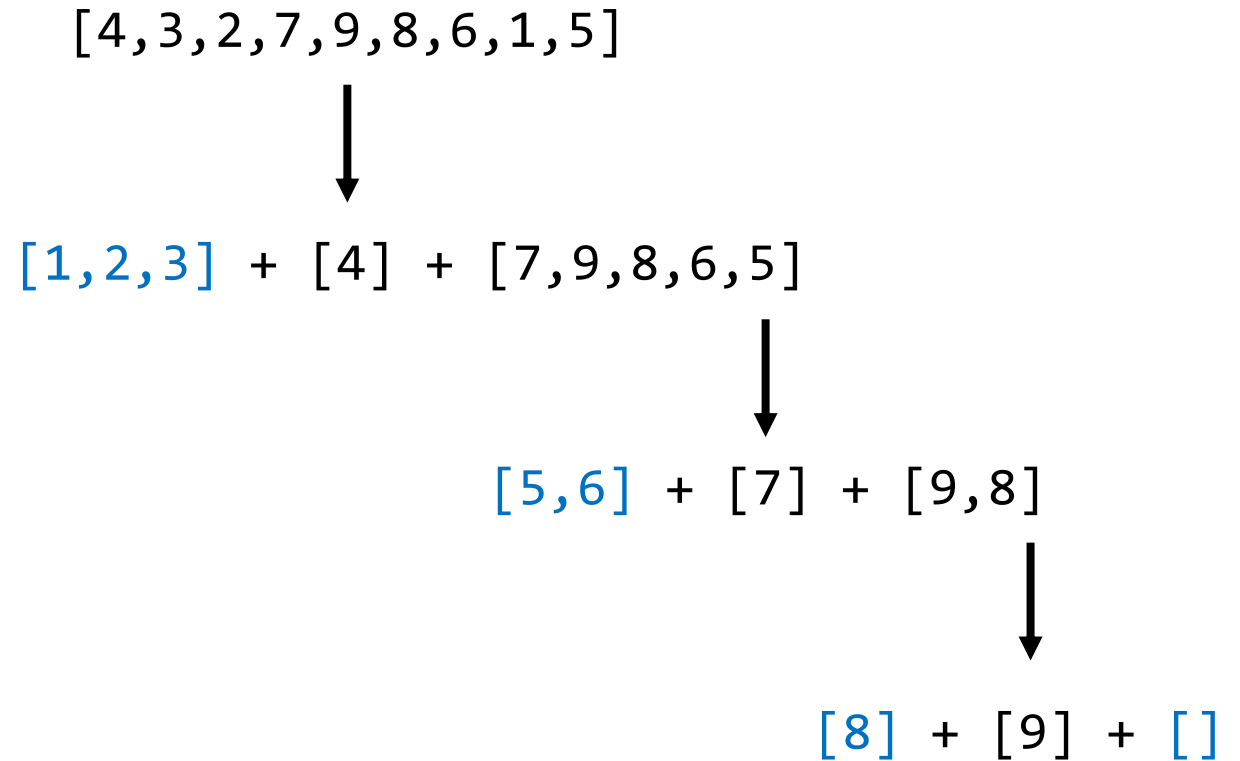
    pivot = items[0] # choose pivot as first item
    low = []
    high = []

    # sort the items into low or high list
    for i in range(1, len(items)):
        if items[i] < pivot:
            low.append(items[i])
        else:
            high.append(items[i])

    # recursively call quicksort on lower and higher
    # return combined result
    return quicksort(low) + [pivot] + quicksort(high)

list1 = [4,3,2,7,9,8,6,1,5]
quicksort(list1)

```



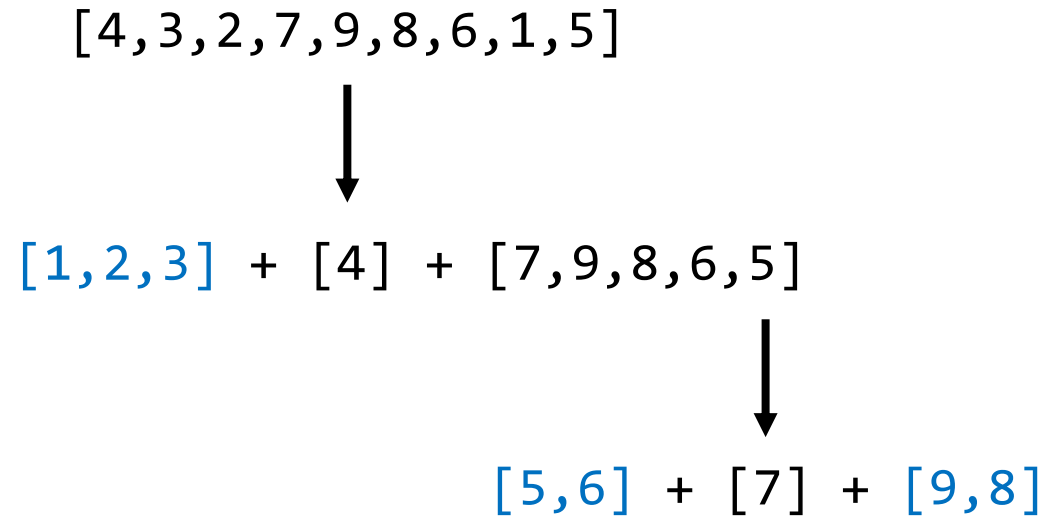
```
def quicksort(items):
    if len(items) <= 1: # base case - single item in list
        return items

    pivot = items[0] # choose pivot as first item
    low = []
    high = []

    # sort the items into low or high list
    for i in range(1, len(items)):
        if items[i] < pivot:
            low.append(items[i])
        else:
            high.append(items[i])

    # recursively call quicksort on lower and higher
    # return combined result
    return quicksort(low) + [pivot] + quicksort(high)

list1 = [4,3,2,7,9,8,6,1,5]
quicksort(list1)
```



```
def quicksort(items):  
    if len(items) <= 1: # base case - single item in list  
        return items  
  
    pivot = items[0] # choose pivot as first item  
    low = []  
    high = []  
  
    # sort the items into low or high list  
    for i in range(1, len(items)):  
        if items[i] < pivot:  
            low.append(items[i])  
        else:  
            high.append(items[i])  
  
    # recursively call quicksort on lower and higher  
    # return combined result  
    return quicksort(low) + [pivot] + quicksort(high)  
  
list1 = [4,3,2,7,9,8,6,1,5]  
quicksort(list1)
```

[4,3,2,7,9,8,6,1,5]
↓
[1,2,3] + [4] + [7,9,8,6,5]

```
def quicksort(items):
    if len(items) <= 1: # base case - single item in list
        return items

    pivot = items[0] # choose pivot as first item
    low = []
    high = []

    # sort the items into low or high list
    for i in range(1, len(items)):
        if items[i] < pivot:
            low.append(items[i])
        else:
            high.append(items[i])

    # recursively call quicksort on lower and higher
    # return combined result
    return quicksort(low) + [pivot] + quicksort(high)

list1 = [4,3,2,7,9,8,6,1,5]
quicksort(list1)
```

[4,3,2,7,9,8,6,1,5]